

The Code Barrier: What LLMs Actually Understand?

Serge Lionel NIKIEMA
University of Luxembourg
Luxembourg, Luxembourg
lionel.nikiema@uni.lu

Jordan Samhi
University of Luxembourg
Luxembourg, Luxembourg
jordan.samhi@uni.lu

Abdoul Kader Kaboré
University of Luxembourg
Luxembourg, Luxembourg
abdoulkader.kabore@uni.lu

Jacques Klein
University of Luxembourg
Luxembourg, Luxembourg
jacques.klein@uni.lu

Tegawendé F. Bissyandé
University of Luxembourg
Luxembourg, Luxembourg
tegawende.bissyande@uni.lu

ABSTRACT

Understanding code represents a core ability needed for automating software development tasks. While foundation models like LLMs show impressive results across many software engineering challenges, the extent of their true semantic understanding beyond simple token recognition remains unclear. This research uses code obfuscation as a structured testing framework to evaluate LLMs’ semantic understanding capabilities. We methodically apply controlled obfuscation changes to source code and measure comprehension through two complementary tasks: ① generating accurate descriptions of obfuscated code and ② performing deobfuscation, a skill with important implications for reverse engineering applications.

Our testing approach includes 13 cutting-edge models, covering both code-specialized (e.g., StarCoder2) and general-purpose (e.g., GPT-4o) architectures, evaluated on a benchmark created from CodeNet and consisting of filtered 250 Java programming problems and their solutions. Findings show a statistically significant performance decline as obfuscation complexity increases, with unexpected resilience shown by general-purpose models compared to their code-focused counterparts. While some models successfully identify obfuscation techniques, their ability to reconstruct the underlying program logic remains constrained, suggesting limitations in their semantic representation mechanisms. This research introduces a new evaluation approach for assessing code comprehension in language models and establishes empirical baselines for advancing research in security-critical code analysis applications such as reverse engineering and adversarial code analysis.

1 INTRODUCTION

The rapid adoption of Large Language Models (LLMs) in software engineering has created a paradox: while these systems can generate syntactically correct code and emulate programming patterns with impressive fluency [6, 48, 53], fundamental questions about their true comprehension capabilities remain unanswered. Behind the seemingly successful performance on standard benchmarks lies a critical uncertainty—*are these models genuinely understanding code semantic, or reconstructing patterns encountered during training?*

Code comprehension—the ability to grasp the underlying purpose, logic, and semantics of code—is a fundamental capability for automating software engineering processes [24, 42]. While recent studies have evaluated LLMs on various code-related tasks [2, 14, 23], these evaluations typically involve standard, well-formatted

code that closely resembles patterns in their training data. Such evaluations may not effectively distinguish between genuine semantic understanding and sophisticated pattern recognition [4, 26].

This paper introduces a different perspective on assessing LLMs’ code comprehension by leveraging code obfuscation as a challenging evaluation paradigm. Code obfuscation intentionally transforms code to hinder human comprehension while preserving functionality [7, 20]—creating an adversarial scenario that forces models to demonstrate deeper semantic understanding rather than relying on superficial patterns. Unlike previous work that focused on LLMs’ performance on standard programming tasks [6, 53], our study specifically targets the models’ resilience to obscured code structures and their ability to recover semantic meaning despite syntactic distortion.

Recent work explored LLMs’ capabilities for security-related issues [32], notably vulnerable code generation, but only few have touched on obfuscated code identification [11]. Our work differs significantly by: ① systematically applying graduated levels of obfuscation complexity to assess comprehension degradation, ② directly comparing code-specialized and general-purpose LLMs across identical tasks, and ③ evaluating both explanation and deobfuscation capabilities as complementary measures of semantic understanding.

With our study, we aim to provide deeper insights into the true nature of LLMs’ code comprehension. This understanding is critical for several reasons: First, it helps establish realistic expectations about LLMs’ abilities, in particular in security-critical applications where adversarial code analysis is essential [32]. Second, it identifies limitations in current models that inform future research and development [5]. Finally, it establishes a benchmark for measuring progress in LLMs’ semantic code understanding over time [39].

Our contributions include:

- ① identifying obfuscated code analysis as a relevant and yet less-studied task to probe LLMs for code comprehension;
- ② an empirical study of 13 state-of-the-art LLMs across diverse obfuscation techniques and complexity levels;
- ③ a comparative analysis revealing unexpected strengths of general models over code-specialized ones on obfuscated code;
- ④ insights into LLMs’ practical applications and limitations for deobfuscation tasks in reverse engineering contexts.

2 BACKGROUND & RELATED WORK

This section provides relevant background on LLMs, current research on code comprehension, and obfuscation techniques.

2.1 LLMs: Capabilities and Limitations

LLMs have demonstrated remarkable capabilities across various domains. In software engineering, models like GPT-4 [29], LLaMA [47], and CodeX [6] have shown promising results on tasks such as code completion [44], bug fixing [48], and code translation [40].

These models generally leverage the transformer architecture [50] and self-supervised learning on vast corpora of code, enabling them to capture patterns and relationships in programming languages. Code LLMs, which are code-specialized models such as CodeLlama [41] and DeepSeek-Coder [8], are typically either pre-trained directly on code repositories or fine-tuned from general-purpose LLMs using programming-specific datasets [53].

Despite their successes, LLMs exhibit several notable limitations. First, they are susceptible to hallucinations—generating plausible but incorrect or nonsensical outputs [17]. Second, they often struggle with complex reasoning tasks that require multi-step logical inference [51], which is particularly problematic for code understanding. Third, their performance degrades significantly on out-of-distribution examples [55], which suggests reliance on superficial patterns rather than deeper comprehension.

A fundamental challenge in evaluating LLMs for code understanding lies in distinguishing between genuine semantic comprehension and sophisticated pattern matching [4]. Most existing benchmarks [2, 14, 23] evaluate models using code samples that closely resemble those in training data, potentially rewarding memorization rather than understanding. As a result, assessing true code comprehension capabilities requires novel evaluation methodologies that can probe beyond surface-level pattern recognition.

2.2 Studies on LLMs and Code Comprehension

Code comprehension research with LLMs has evolved across several dimensions. Early work focused primarily on syntax-level understanding and code completion tasks [38, 44]. These studies demonstrated LLMs can predict tokens and complete partial programs but provided limited insight into deeper semantic understanding.

More recent research has explored LLMs’ capabilities for higher-level code understanding tasks. Studies by Peng et al. [33] and Jain et al. [16] evaluated models’ ability to generate natural language explanations of code functionality, finding that while LLMs can produce plausible explanations for simple programs, their performance deteriorates with increasing program complexity. Tufano et al. [49] investigated LLMs’ capabilities for code summarization, demonstrating moderate success but noting significant challenges with larger, more complex codebases.

Several studies have addressed code analysis capabilities of LLMs. Prenner and Robbes [35] examined how LLMs could detect code smells and suggest refactorings, while Pearce et al. [32] evaluated their ability to identify security vulnerabilities. Both found that while models showed promising abilities, they missed important issues that human experts would identify. Fang et al. [11] conducted one of the first studies on LLMs’ obfuscation capabilities.

Despite this growing body of research, a comprehensive assessment of LLMs’ true code comprehension capabilities remains elusive. Most studies have focused on standard, well-formatted code rather than challenging models with adversarial examples designed to test deeper comprehension. In addition, comparative analyzes

between different model architectures and sizes on identical comprehension tasks have been limited.

2.3 Code Obfuscation Techniques

Code obfuscation refers to deliberate modifications that make source code difficult for humans to understand while preserving its functionality [7, 20]. The motivation of obfuscating software is twofold: ① to prevent reverse engineering and protect intellectual property; and ② to hide malicious code from analysts. While numerous obfuscation techniques exist, our study focuses on three categories that represent different dimensions of code manipulation.

2.3.1 Lexical Obfuscation: Variable Renaming. Variable renaming is one of the most common and straightforward obfuscation techniques, targeting the readability of code at the lexical level [3]. This technique involves replacing meaningful identifier names with uninformative alternatives, which eliminates the semantic hints that programmers typically embed in their naming conventions (e.g., replacing variable *accountBalance* with *a*).

While variable renaming preserves program semantics entirely, it significantly impairs comprehension by eliminating the domain knowledge embedded in identifier names. Studies by Feitelson et al. [12] have shown that meaningful variable names can improve code comprehension by up to 30%, making their removal a powerful obfuscation strategy. The idea with this investigation is to assess to what extent LLMs rely on syntactic namings.

2.3.2 Structural Obfuscation: Dead Code Injection. Dead code injection alters program structure by adding syntactically valid but semantically irrelevant code segments that are never executed or have no effect on program behavior [34]. It increases program complexity and obscures control flow and logic by introducing “noise”.

Dead code injection significantly increases the cognitive load required for program comprehension by forcing analysts to determine which code segments are relevant to the program’s functionality [28]. This technique is particularly effective at confusing both human analysts and automated tools because it requires distinguishing between functional and non-functional code. The idea of this investigation is to assess whether LLMs actually perceive code structure and can disambiguate relevant parts for code behavior.

2.3.3 Semantic Obfuscation: Integer and String Encryption. Integer and string encryption represent semantic obfuscation techniques that transform literal values in the code into encrypted or encoded forms that are decrypted or decoded at runtime [19]. These techniques hide the actual values used in the program, making it more challenging to understand the program’s purpose and behavior.

Unlike variable renaming, which preserves code semantics while changing names, or dead code injection, which adds irrelevant code, encryption transforms the data while maintaining equivalence through runtime decryption [1]. This creates a significant challenge for comprehension as it obscures the values in the program, requiring mental execution of the decryption logic to understand program’s behavior.

These three categories of obfuscation—lexical, structural, and semantic—represent complementary approaches. These constitute a solid foundation for evaluating LLMs’ ability to comprehend code despite intentional interferences. The deliberate opacity introduced

by obfuscation creates an ideal test bed for evaluating true code comprehension. Unlike standard code examples, obfuscated code forces any analyzing entity—human or AI—to look beyond surface syntax and recover semantic despite intentional interference.

3 METHODOLOGY

This research investigates whether LLMs can comprehend obfuscated code. To address this question, we evaluate LLMs on two specific tasks: ❶ explaining obfuscated code; and ❷ generating de-obfuscated code. In this study, we consider the Java programming language as our usage scenario.

3.1 Research questions

We aim to answer the following research questions (RQs) to assess LLMs’ capabilities to understand the semantics of code:

RQ1: To what extent can LLMs accurately describe the semantic purpose of code beyond surface-level syntax? In this RQ, we investigate also how significantly do different levels of code obfuscation impact LLMs’ ability to discern a program’s underlying functionality.

RQ2: How effectively can current LLMs deobfuscate code?

3.2 Benchmarks

To evaluate LLMs on code semantic understanding, multiple benchmarks are required: ❶ first, we need Java source code along with the description of the code; ❷ second, we need Java source code along with corresponding input–output examples; and ❸ lastly, we need Java source code along with corresponding sets of obfuscated versions at different levels. We outline our process for constructing our dataset, OBSCURA, below.

Table 1: Dataset Comparison

	Code Java	Description	Executable	Input/Output	Multiple Solutions
CodeNet[36]	✓	✓	✓	✓	✓/✓/✓
CodeSearchNet[15]	✓	✓	×	×	×
LeetCode[57]	✓	✓	✓	✓	✓
IJA Dataset[10]	✓	✓	×	×	×
XLCost[13]	✓	✓	✓	✓	✓
CodeXGlue[23]	✓	✓	✓	✓	✓
FunCom[21]	✓	✓	×	×	×

3.2.1 Initial Dataset. We have relied on the CodeNet [37] dataset as our base to build the three datasets we previously described. Several other datasets could have been suitable, but CodeNet meets all the criteria we previously described and, as shown in Table 1, it offers multiple implementations for each problem, paramount for our needs, as we show in this Section.

The CodeNet project is a huge dataset of 14+ million code samples spanning 55 programming languages, created to advance AI code research. Sourced from online judge platforms (e.g., AIZU and AtCoder), it contains 4053 programming problems.

CodeNet is made of specialized benchmarks such as C++1000, C++1400, Python800, or Java250. For our research, we have considered the **Java250** benchmark that contains 250 programming problems with more than 75 000 possible implementations (i.e., 300 implementations on average per problem). Note that, for our study, we identified and retained both the most complex implementation

of a given problem based on the cyclomatic complexity metric [27], and the less complex implementation. Furthermore, we also retained two versions of the source code: ❶ the original source code from CodeNet; and ❷ the same piece of code without any comment.

Eventually, our initial dataset, OBSCURA, is a set of algorithmic problems (e.g., implementing a sorting algorithm), where each problem P consists of $P = (d, I, s_c^+, s_c^-)$, where d represents the description of the problem, I is the set of input–output pairs, s_c^+ is the most complex implementation of the given problem, s_c^+ is s_c^+ without the comments in the code, and s_c^- is the less complex implementation of the given problem.

3.2.2 Code Description. For a given problem $P \in \text{OBSCURA}$, its description d describes a programming task in the form of a narrative story (e.g., Batman needs to sort a list of Gotham’s most-wanted criminals by threat level, ensuring he takes down the most dangerous ones first.), which might involve specific context, characters, etc. Moreover, they contain numerical values, constraints, and mathematical formulas. As a result, their meaning depends on specific scenarios unrelated to algorithmic logic. This constitutes an issue to semantically compare LLM-generated descriptions since LLMs would describe the essence of the code without context (e.g., it cannot guess that Batman was actually used to describe the algorithmic problem in a given dataset).

To alleviate this issue, we propose the following solution: we aim at reformulating any problem description d into another description d' that focused on algorithmic logic. The goal is to maintain a neutral descriptive structure for fair comparison. To do so, we use an LLM-based approach to automate the process as well as manual checking to assess the result. More specifically, for a given problem P , we feed GPT-4o with both d and s_c^- and ask it to provide free-of-context and descriptive exercise statements d' .

This is an example of our prompting strategy to get the free-of-context description of a given problem:

Prompt 1

"I will provide you with a coding exercise and a corresponding solution. Your task is to analyze both and create a new, beginner-friendly coding exercise. This new exercise should be generic, context-free, and clearly describe the desired functions. Focus on the fundamental programming concepts being practiced and ensure the goal of the exercise is easily understood. It must contain the title of the exercise, the problem statement and the requirement. Do not add example, inputs, outputs. Do not exceed 512 tokens."

We have **manually validated all the 250 GPT-4o-generated free-of-context descriptions**. Among these, 27 did not precisely describe s_c^- . For these problems, we manually generated the description of the code. Then, to confirm the similarity of the free-of-context GPT-4o generated description, for each problem P , we provide both GPT-4o and Claude Sonnet 3.7 d and d' and ask them to provide a similarity score range from 0 to 1. Here is an example of the prompt we have used:

Prompt 2

"You are given two descriptions of code snippets without the code. Provide a similarity score between 0 and 1, where 1 means that the two exercises are similar in terms of goal, constraints, and requirements, and 0 means they solve completely dissimilar."

For the 250 pairs of d and d' , the similarity score ranges from 0.9 to 1. This means that the reformulated exercise statement kept the meaning and the intention of the original exercise statement. At the end, we embellish OBSCURA where each P now consists of $P = (d, d', \mathcal{I}, s_c^+, s^+, s_c^-)$.

3.2.3 Obfuscation. To obfuscate the code samples of our dataset into, we employed Obfuscator, an open-source Java obfuscation tool [43]. Obfuscator allows users to select the desired type of obfuscation with different levels of granularity. For our experiments, we have separately consider the following obfuscation types: ❶ variable renaming, which replaces meaningful names with non-descriptive ones which reduces readability while preserving functionality, ❷ dead code injection, which adds non-functional code to obscure comprehension without affecting execution, and ❸ encryption of integer and string literals, which obfuscates literals by replacing them with decryption calls or complex expressions, which increases code complexity while preserving functionality. For each problem $P \in \text{OBSCURA}$, we considered s^+

and did the following:

❶ we obfuscated s^+ with variable renaming as s_{ren}^+ ; ❷ we obfuscated s^+ with dead code injection as s_{dead}^+ ; and ❸ we obfuscated s^+ with encryption of integer and string literals as s_{enc}^+ . We, again, embellished OBSCURA with this information. Eventually, our dataset OBSCURA, comprises 250 samples, each described as $P = (d, d', \mathcal{I}, s_c^+, s^+, s_c^-, s_{ren}^+, s_{dead}^+, s_{enc}^+)$.

3.3 Large Language Models

In our study, we selected various LLMs spanning three main categories. The first category consists of pre-trained *generalist* LLMs, such as GPT-4o, which are designed for broad applications across multiple domains. The second category includes *fine-tuned* LLMs that have been specifically optimized for code-related tasks based on pre-trained foundations, like CodeLlama. The third category comprises *specialized* LLMs that were purpose-built from the ground up for code-related applications. In total, we evaluate 13 distinct LLMs across these three categories. They are presented in Table 2.

Table 2: Selected Large Language Models

ID	LLMs	Weight	Generalist	Fine-tuned	Specialized
1	GPT-4o[30]	1760	✓		
2	GPT-3.5-turbo[56]	175	✓		
3	DeepSeek-R1-Distill-Qwen-7B[9]	16	✓		
4	mistralai/Mistral-7B-Instruct-v0.2[18]	16	✓		
5	gemma-2-9b-it[46]	9	✓		
6	Llama-3.1-8B[31]	8	✓		
7	Qwen2.5-7B-Instruct[54]	7	✓		
8	CodeLlama-7b-hf[41]	7		✓	
9	codegemma-7b-it[45]	7		✓	
10	Mamba-Codestral-7b-v0.1 [codestral]	7		✓	
11	DeepSeek-Coder-V2-Lite-Instruct[8]	7		✓	
12	starchat2-15b-v0.1[22]	15			✓
13	starcode2-15b-instruct-v0.1[52]	15			✓

3.4 Evaluation Tasks

To assess LLMs' understanding of code semantics, we designed three evaluation phases: ❶ generating code descriptions with LLMs and comparing them to our ground truth (Section 3.4.1); ❷ generating descriptions for obfuscated code with LLMs and comparing them to our ground truth (Section 3.4.2); and ❸ using LLMs to de-obfuscate code and verifying its semantics based on input-output pairs from our ground truth (Section 3.4.3).

3.4.1 Description generation task. The code can be in one of two configurations when prompting an LLM for its description: ❶ the code contains comments; and ❷ the code does not contain any comments. For this task, we aim at assessing LLMs into generating descriptions in these two configurations and investigate the impact of comments in code samples for LLM code comprehension.

However, not all of the 250 samples in OBSCURA contain code with comments. Specifically, not all s_c^+ actually include comments; rather, these samples were not pre-processed to remove comments where present. As a result, to study the impact of comments, for this task, we only retain any problem $P \in \text{OBSCURA}$ for which s_c^+ actually contains comments, i.e., 139 samples out of 250.

To generate the descriptions of Java code samples, for each of the 139 problems, we prompted each of the 13 LLMs previously described in Section 3.3 twice: ❶ once with s_c^+ , and ❷ once with s^+ .

The prompt is as follows:

Prompt 3

"Your task is to generate a short summary description for a given piece of code. Your description should focus on how the code addresses the problem it was designed to solve. Here is how to proceed: First, you will be presented with a code snippet:

{New Code}

Next, you will be given an example of code following to his associate problem statement: **{Java code} And {Description}** Analyze carefully the **{New Code}**, paying attention to its structure, functions and logic. Consider How each part of the code contributes to solving the given problem. Present only your summary description."

In this prompt, **{New Code}** is the piece of code which we want to generate a description by the LLM (i.e., s^+ , and s_c^+), **{Java Code}** is another piece of code serving as an example for the LLM along with its description **{Description}** to guide the LLM.

Next, for each problem P , we compare each LLM-generated description to d' using the same prompt as in Section 3.2.2. That is to say, we ask both GPT-4o and Claude Sonnet 3.7 for a similarity score between 0 and 1. If any of the LLM yields a score below 0.5, we consider the LLM-generated description as not accurately describing s^+ or s_c^+ .

For this task, we used the following prompt:

Prompt 4

"You are given two descriptions of code snippets without the code. Provide a similarity score between 0 and 1, where 1 means the two description is about code that solve the same problem,

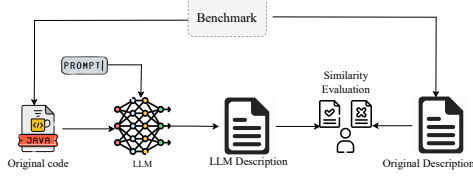


Figure 1: Description generation process and comparison against ground-truth

and 0 means they solve completely different problem. Respond with only a numerical value between 0 and 1."

The overview of this process is shown in Figure 1. We present the results in Section 4.1.

3.4.2 Obfuscated code description generation task. To generate descriptions of obfuscated Java samples, for each problem $P \in \text{OBSCURA}$, we prompt LLMs to first deobfuscate s_{ren}^+ , s_{dead}^+ , s_{enc}^+ and then GPT-4o to generate a description for each of the deobfuscated samples. The prompt is as follows:

Prompt 5

You are an expert in reverse engineering. You will be given an obfuscate code. Your task is to generate a readable and well formatted version of this code.
First, analyze thoroughly the obfuscated code to understand its structure and functionality.

{Obfuscated Code}

Generate a deobfuscated version using meaningful variable and method names, apply proper indentation, follow Java naming conventions, simplify complex expressions, remove redundant code, maintain the original functionality.

Return only the readable code.

Next, for each LLM-generated description of deobfuscated code, we use the same prompt as in Section 3.4.1. That is to say, we ask both Claude Sonnet 3.7 and Deepseek-reason for a similarity score between 0 and 1. If any of the LLM yields a score below 0.5, we consider the LLM-generated description as not accurately describing s^+ . We present the results in Section 4.2.

3.4.3 Deobfuscation Task. To assess LLMs' ability to derive meaningful and readable versions of obfuscated code, we reuse the deobfuscated Java samples previously described in Section 3.4.2.

Then, for each problem P , we compared each LLM-generate deobfuscated piece of code and perform the following steps: ❶ we attempt to compile the deobfuscated code; ❷ if compilation succeeds, we attempt to execute it; ❸ if the program does not crash during execution, we use our set of input-output pairs $\mathcal{I}$ to run the program with the inputs and collect the outputs; and ❹ we compare the generated output (after execution) with the expected outputs from our ground-truth $\mathcal{I}$. The overview of this process is shown in Figure 2. We present our results in Section 4.2.

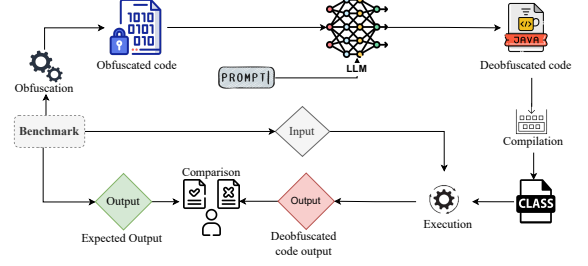


Figure 2: Deobfuscation process

3.5 Evaluation Metrics

During this study, we employed several metrics to evaluate the effectiveness of LLMs in generating descriptions and their ability to provide deobfuscated code:

- **Cyclomatic Complexity:** We tracked the evolution of code complexity before and after obfuscation, providing a quantitative measure of structural changes.
- **LLM-Based Semantic Similarity:** To overcome limitations of traditional methods like embedding with vector similarity computation, we implemented an LLM-based measurement approach. We use three models to evaluate the semantic similarity between text (i.e., GPT-4o for description generation of deobfuscated code, and Deepseek-reason and Claude for evaluating descriptions semantic accuracy).
- **Statistical Analysis:** We employed the Mann-Whitney-Wilcoxon (MWW) [25] test to measure the statistical differences between samples where LLMs failed to generate adequate descriptions versus the original dataset, analyzing differences in complexity, lines of code, and token count.

As a supplementary validation measure, we prompted these same two models to provide binary (yes/no) classifications regarding whether the generated descriptions were programming-related. This additional step helped verify the fundamental relevance of the generated content to the programming domain.

4 EXPERIMENTAL RESULTS

This section presents the experimental results of our study.

4.1 Baseline Analysis: Comment and Language Effects on LLM Code Description Generation

In this section, we present the results of our evaluation of LLM-generated descriptions on non-obfuscated code. Section 4.1.1 investigates the impact of comments, and Section 4.1.2 investigates the impact of the language used in comments.

4.1.1 Impact of Comments in Code for LLM Code Comprehension.

Goal: We investigate how comment tokens influence LLMs when generating code descriptions. Despite comments being relatively modest in size in our dataset, they may significantly impact the LLMs' outputs compared to relying solely on the lexical tokens of the code itself. We assess the relative weight LLMs assign to natural language explanations versus code structure and syntax.

Experiments: We identified 101 problems in our dataset where the implementation (s_c^+) contains comments in the English language. For these problems, we prompted each LLM to generate descriptions for both the commented version (s_c^+) and the version with comments removed (s^+). We then evaluated the accuracy of these descriptions using the process described in Section 3.2.2.

Results: Figure 3 presents the percentage of accurate descriptions generated by each LLM under both conditions. We observe:

- (1) For 8 out of 13 LLMs tested, the performance in generating accurate descriptions is higher when considering code with comments. The average improvement was 3.7%, with Deepseek-R1 showing the largest gain (11.5%).
- (2) GPT-4o drastically outperforms all other LLMs tested in our study, achieving around 87% accuracy with comments and around 84% without comments. The differences in performance distributions are however not statistically significant (MWW).
- (3) There is a striking performance disparity across models, with some code-specialized models surprisingly underperforming compared to general-purpose models. Notably, several models (Mistral, CodeLlama, CodeStral, StarChat) exhibit the counterintuitive pattern of performing better with code without comments, showing although only small improvements of 2.9, 5.0, 2.9, and 2.2 percentage points, respectively.

We speculate that this performance variation might still reflect fundamental differences in model training and/or architecture. Models showing improvement with comments likely leverage the natural language explanations as additional context, while those performing better without comments may have developed stronger representations of code syntax and structure during training, potentially being distracted by comments that don't align with their learned patterns. However, given the complexity of these models and the limited scope of our experiment, multiple alternative explanations may exist, including differences in tokenization approaches, varying sensitivities to comment placement, etc.

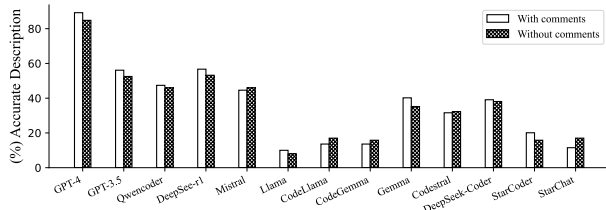


Figure 3: LLMs effectiveness in code description (with or without comments)

[Finding 1] Our empirical results show general-purpose LLMs outperforming code-specialized models in description accuracy. This challenges the common assumption in code-specialized model development that domain-specific training necessarily enhances performance on code tasks. The data suggests that foundational model quality and/or pre-training diversity may be more significant factors than domain specialization for code comprehension tasks. On the studied benchmarks, the effect of comments was not shown statistically significant, potentially due to the fact that these were not Javadoc-style documentation, but rather relatively-short comments (cf. Figure 5).

4.1.2 Impact of Comment Language on LLM Code Understanding.

Goal: We investigate how the natural language (e.g., English) in which code comments are provided affects LLMs' code comprehension abilities. While programming languages are standardized, comments can be written in any natural language.

Experiments: We isolated from our dataset all problems whose reference solutions are associated with Japanese comments. Then we used GPT-4o to translate the Japanese comments to English. We then compared the accuracy of descriptions generated from code with the original Japanese comments versus code with the translated English comments.

Additionally, we analyzed the comment density (ratio of comment lines to total code lines) in the original sets of problems with English and Japanese¹ comments to better understand potential differences in documentation practices.

Results: Figure 4 shows the results of our experiment with Japanese comments. Six LLM models show improved performance when processing code with translated comments compared to the original Japanese comments. Among these, Deepseek-R1 demonstrated the most significant improvement, with a 37.5% increase in effectiveness when working with translated comments compared to the original Japanese comments.

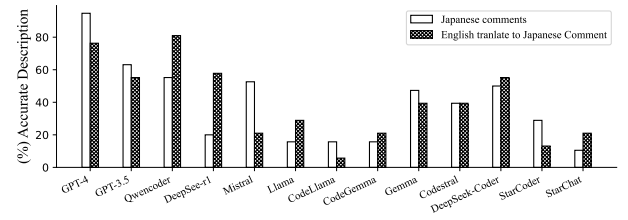


Figure 4: Effectiveness of LLMs to generate descriptions on problems with comments (Japanese vs English translations).

Figure 5 shows the comment density distribution in both sets of problems. Code samples with Japanese comments (after translation) had significantly higher comment ratios—up to 25% of total lines—compared to samples with native English comments, which peaked at approximately 7-8%. This difference in documentation density also likely contributed to the substantial performance improvements observed after translation, as more comprehensive comments provided richer context for the models to leverage.

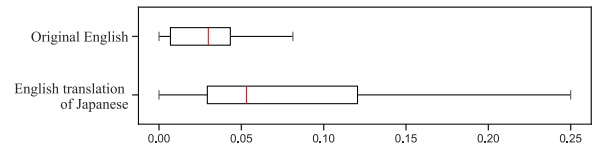


Figure 5: Comment to code Ratio-English vs Japanese (translated in English)

¹We translate the Japanese comments into English to ensure unbiased comparison given lexical differences.

🔗 **[Finding 2]** Our results reveal nuanced patterns in how comment language affects LLMs: (1) Models show varied responses to comment language—some perform better with original Japanese comments (GPT-4, GPT-3.5, Mistral, QwenCoder), while others improve with English translations (DeepSeek-r1, Llama, CodeGemma, StarChat); (2) Japanese comments contain significantly higher comment density (median 10% vs 5% in English), suggesting potential cultural differences in documentation practices. These findings highlight both the diverse multilingual capabilities of different LLMs and the importance of comment comprehensiveness in code understanding tasks.

4.2 Effect of Obfuscation on LLM Code Description Generation

In this section, we present our evaluation of how different obfuscation techniques affect LLMs' ability to generate accurate code descriptions. Section 4.2.1 assesses performance across obfuscation types, Section 4.2.2 investigates characteristics of consistently misunderstood code, and Section 4.2.3 examines LLM failures' nature.

4.2.1 LLM-Generated Description Performance of Obfuscated Code.

Goal: We investigate how different obfuscation types affect LLMs' ability to generate accurate code descriptions. This comparison is paramount because obfuscation targets different aspects of code (i.e., lexical, structural, semantic), allowing us to isolate which code properties LLMs rely on most when processing programs.

Experiment: For each problem in OBSCURA, we prompted the 13 LLMs to generate descriptions for each obfuscated variant: variable renaming (s^+_{ren}), dead-code-injected (s^+_{dead}), and encrypted-literals (s^+_{enc}). We evaluated description accuracy using the same process as for non-obfuscated code. By comparing performance between obfuscated and non-obfuscated variants, we can quantify how each obfuscation type impacts LLMs' description capabilities. **Results:** Figure 6 presents the accuracy of descriptions generated for each obfuscation technique compared to the non-obfuscated baseline. We observe consistent performance degradation across most models, with technique-specific patterns:

Variable Renaming caused significant performance degradation across almost all models. The average accuracy dropped by 18.6 percentage points, with some models experiencing declines of over 30 percentage points. This pronounced sensitivity to identifier names suggests LLMs heavily rely on lexical semantics embedded in variable names rather than structural code understanding. GPT-4o showed the most resilience, with only a 7.3% decline.

Dead Code Injection had, surprisingly, the least impact on description accuracy, with an average decline of only 6.2%. Several models (QwenCoder, Mistral, and GPT-4o) showed minimal performance changes, suggesting they can effectively identify and filter out non-functional code segments when generating descriptions.

Literal Encryption caused substantial performance degradation (average decline of 21.4%), particularly affecting general-purpose models. Even GPT-4o experienced a significant drop of 19.9%, though it remained the top performer with 58.8% accuracy on encrypted code. This suggests that obfuscating literal values, associated with the introduction of complex code for run-time decryption, severely impairs LLMs' ability to infer program semantics.

QwenCoder showed unusual resilience to variable renaming and dead code injection compared to other models, maintaining nearly identical performance across these conditions. This outlier behavior warrants investigation into its training data and/or architecture.

🔗 **[Finding 3]** LLMs exhibit critical dependencies on semantic elements: meaningful variable names and unobfuscated literals. Description accuracy drops significantly when these human-readable components are altered, while structural obfuscation through dead code has less impact. This suggests current models process code primarily through surface-level linguistic patterns rather than deeper algorithmic structures independent of representation.

4.2.2 Why LLMs Fail with the Code Description Generation Task.

Goal: We investigate the reason why LLMs struggle to generate code descriptions for obfuscated code and non-obfuscated.

Experiment: We identified 13 samples where all LLMs failed to generate accurate descriptions across all conditions (non-obfuscated and all three obfuscation variants). For these consistently misunderstood samples, we calculated cyclomatic complexity and token count, then compared their distributions against the entire dataset using the Mann-Whitney U test to determine statistical significance. **Results:** Figure 7 shows the distribution of cyclomatic complexity and token count for both the failed samples and the original dataset. The analysis reveals a striking pattern:

Failed samples exhibit higher average and median cyclomatic complexity values (38.7 vs. 32.5 for average and 31 vs. 37 for median) while containing substantially fewer tokens (405.5 vs. 664.0 on average and 609 vs. 364 in terms of median). The token distribution in failed samples also shows a narrower range (standard deviation 171.4 vs. 336.5). The Mann-Whitney U test confirmed statistical significance for token count differences (p-value = 0.007), though the complexity difference was not statistically significant. Nevertheless, the perceived inverse relationship between complexity and code size suggests that LLMs struggle most with code that packs complex logic into compact implementations. These dense code structures, which do "more with less", present a particular challenge for current model architectures.

🔗 **[Finding 4]** Samples with high information density—implementing complex logic with fewer tokens—present a unique challenge for current LLM architectures. This finding suggests a blind spot in model capabilities when processing code that combines brevity with logical complexity.

4.2.3 Do LLMs Hallucinate or Misinterpret Code?

Goal: We investigate to what extent LLMs generate description content that is entirely unrelated to coding, such as a poem, which would indicate hallucination.

Experiment: To achieve our goal, we relied on GPT-4o and prompted it for all descriptions generated by all LLMs on all obfuscated variants of the 250 Java samples. The prompt is as follows:

Prompt 6

"Determine if the given code description is programming-related or not. Answer with only 'yes' or 'no'."

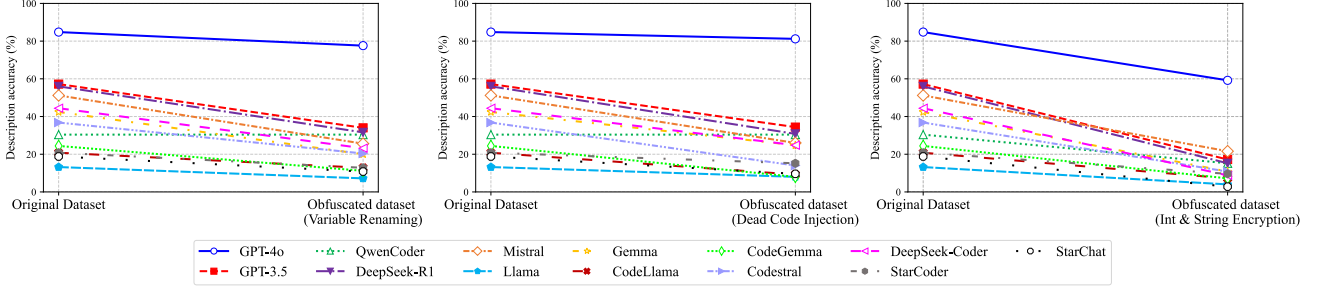


Figure 6: Impact of obfuscation techniques on description accuracy compared to non-obfuscated code - comments are removed from all samples in the original dataset and before any obfuscation technique is applied.

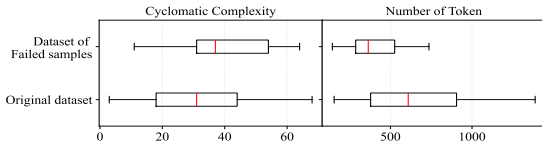


Figure 7: Distribution of cyclomatic complexity and token counts for samples where all LLMs failed to generate an accurate description, compared to the entire dataset.

The rationale is as follows: If the generated description is related to code, it suggests that it simply does not accurately describe the sample, i.e., it is a misinterpretation rather than a complete failure. However, if the description is entirely unrelated to code, it indicates that the model has hallucinated while generating the description.

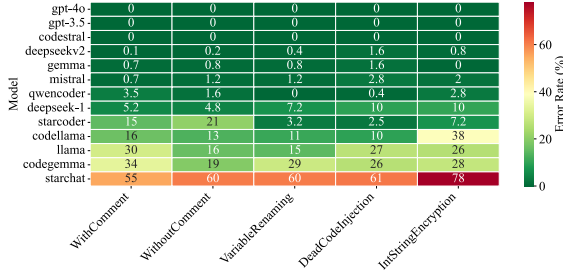


Figure 8: LLMs Error Rate of Generating Code-related Description

Results: Figure 8 presents our results. It includes the error rate, of the studied LLMs, in generating *code-related* descriptions across all code sample configurations. Several models, despite being unable to provide accurate descriptions, consistently generate content that remains within the programming domain, i.e., code descriptions are related to code. This suggests that these models maintain contextual awareness even during failure modes. Conversely, other models demonstrate a tendency to produce descriptions that completely diverge from the programming context when they fail, i.e., the descriptions generated are unrelated to code. An unexpected finding is that one of the models exhibiting the highest rate of contextually

unrelated descriptions is specifically designed and fine-tuned for code-related tasks.

[Finding 5] Our analysis reveals significant variation in how LLMs fail when generating descriptions. Top-performing models (GPT-4o, GPT-3.5, and CodeStral) never produce non-code-related content, while other models show increasing rates of domain-irrelevant outputs. Surprisingly, StarChat—specifically designed for code tasks—exhibits the highest error rates (55-78%) in generating code-related descriptions across all configurations. This suggests fundamental differences in how models maintain contextual awareness under challenging conditions, with some preserving domain relevance despite inaccuracies while others experience catastrophic failures producing completely unrelated content.

4.3 Deobfuscation Capabilities

4.3.1 Can LLMs effectively deobfuscate code?

Goal: We evaluate how effectively LLMs can reconstruct clean, functionally correct code from obfuscated inputs. This capability represents a stronger test of code comprehension than description generation, as it requires not just understanding the code’s purpose but also reconstructing its implementation details.

Experiment: We feed obfuscated code samples to LLMs with a specialized prompt (Prompt 5) requesting deobfuscated versions, then we validate the generated solutions through an automated pipeline that compiles the code, tests it with dataset inputs, and compares outputs against expected results. This approach quantitatively measures how effectively LLMs can extract meaningful, functionally equivalent implementations from deliberately complex code, providing insights into their semantic understanding and code refactoring capabilities.

Results: Figure 9 presents the results of our experiment. Our analysis reveals several important patterns:

Performance hierarchy across obfuscation types: All models show a consistent pattern of decreasing performance according to obfuscation complexity: variable renaming presents the most approachable challenge, followed by dead code injection, with literal encryption proving most difficult. This hierarchy aligns with our findings on description generation tasks, suggesting consistent underlying limitations in LLM code processing.

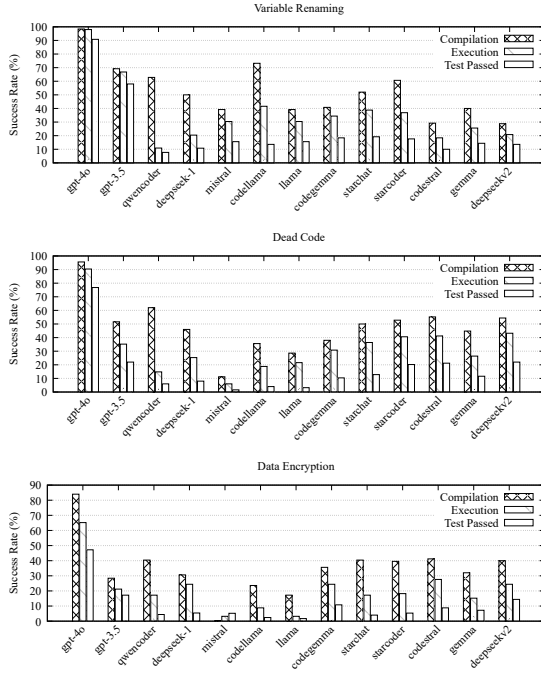


Figure 9: Evaluation of LLMs ability to deobfuscate code

Syntactic vs. semantic success: We observe a substantial gap between compilation rates and functional correctness across all models and obfuscation types. For example, several models achieve 60-70% compilation success on dead code obfuscation but only 40-50% test-passing rates. This indicates that LLMs can often generate syntactically valid code that fails to preserve the original semantics.

Specialized vs. General models: Interestingly, code-specialized models generally outperform general-purpose ones on deobfuscation tasks (except for the GPT family), which contrasts with our findings for description generation tasks. This discrepancy suggests that different aspects of code understanding are captured by these two task types, with deobfuscation potentially benefiting more from exposure to diverse code transformations during training.

Variable renaming resilience: Several top-performing models achieve impressive results on variable renaming deobfuscation, with compilation rates above 60% and test-passing rates reaching 90% for the best models. This suggests that despite the strong impact of variable renaming on description tasks, LLMs can effectively reverse this transformation when explicitly prompted to do so.

🔍 **[Finding 6]** Large Language Models for code struggle with code description tasks. However, they demonstrate superior performance when it comes to deobfuscation.

4.3.2 Semantic Preservation in LLM Deobfuscation. Goal: We evaluate whether LLM-deobfuscated code preserves the semantic intent of the original implementation. Understanding semantic preservation is crucial because functional equivalence alone does not

guarantee that the code’s underlying algorithmic approach and design patterns are maintained.

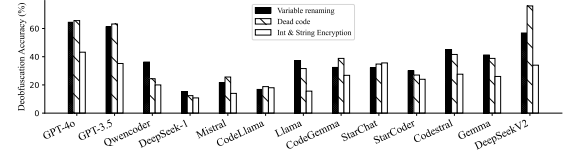


Figure 10: Deobfuscation success rates across different obfuscation techniques

Experiment: We analyzed code successfully deobfuscated by LLMs (code that compiled and passed functional tests) by generating descriptions of both the deobfuscated outputs and the original code using GPT-4o, then comparing their semantic similarity.

Results: Figure 10 shows deobfuscation performance across models and obfuscation types. We observe that models achieving high functional correctness (particularly GPT-4o, DeepSeekV2, and GPT-3.5) still vary significantly in their ability to restore semantic clarity. Variable renaming proved most tractable (22.7% average success), followed by dead code removal (16.3%), with literal encryption presenting the greatest challenge (12.7%). Manual analysis showed that some LLM-deobfuscated code works but does not fully restore original clarity, creating a misleading impression of deobfuscation success when evaluating solely through functional testing.

🔍 **[Finding 7]** Even when LLMs produce deobfuscated code that passes functional tests, it often remains partially obfuscated. The code works but is not fully restored to its original clarity. LLMs are best at renaming variables, decent with numbers, but consistently struggle with decrypting strings. Additionally, specialized code LLMs generally perform better at deobfuscation tasks than general models, with GPT-4o standing as the notable exception that outperforms even specialized code models.

5 DISCUSSION

Throughout our discussion, we recognize that the ability to generate accurate code descriptions is only one indicator of code understanding. We now discuss the key implications and threats to validity.

5.1 Key Findings and Implications

Our study yields six significant findings regarding LLMs’ capabilities in processing obfuscated code:

❶ First, general-purpose LLMs (particularly GPT-4o and Deepseek-R1) outperformed code-specialized models in generating accurate descriptions of obfuscated code. This unexpected result challenges the assumption that domain-specific fine-tuning necessarily improves performance on code-related tasks. Instead, the breadth of training data and base model quality may be more important factors for robust code processing. Tasks related to obfuscated code analysis can therefore be considered as new tasks in the development of more related code-specialized LLMs.

❷ Second, different obfuscation techniques revealed specific dependencies in how LLMs process code when generating descriptions.

Variable renaming caused most significant performance degradation across all models, indicating a strong reliance on meaningful identifier names rather than structural understanding. Models showed surprising resilience to dead code injection, suggesting some ability to filter irrelevant information when producing descriptions. However, integer and string encryption significantly impaired description accuracy, revealing limitations in processing code when literal values are obscured or when the code is inflated with complex code (for dynamic decryption).

③ Third, our analysis of failure cases revealed that LLMs particularly struggle with describing concise yet structurally complex code. This combination of high information density and logical complexity represents a specific blind spot in current model architectures, which has implications for their application to highly optimized or production-grade code.

④ Fourth, description accuracy improvement when translating non-English comments highlights a practical constraint in multilingual code processing. This finding has important implications for international development teams and automated code analysis tools.

⑤ Fifth, while code-specialized LLMs often fail to generate accurate descriptions, they generally outperform general-purpose models on deobfuscation tasks (with the exception of the GPT family). Description generation and deobfuscation appear therefore to engage different aspects of code understanding. Code-specialized models excel at structural transformations but struggle with abstracting and articulating code purpose in natural language.

5.2 Theoretical and Practical Implications

The aforementioned findings suggest that current LLMs process code using a layered approach that includes surface-level pattern recognition, structural analysis, and semantic integration attempts. This processing model explains why no single obfuscation technique completely defeated all LLMs, while also explaining why all models showed some performance degradation in description tasks.

For software engineering practice, our results indicate that:

- Meaningful variable names significantly improve LLMs' ability to process code, potentially even more than removing dead code or maintaining clear control flow.
- LLMs may provide less reliable assistance when working with highly optimized, concise code bases compared to more verbosely written application code.
- Language barriers in comments represent a meaningful constraint for current LLMs, suggesting the value of standardized documentation languages in international teams.

For security applications, the varied effectiveness of different obfuscation techniques provides insights for both software protection and reverse engineering. While LLMs show promise as assistive tools for deobfuscation tasks, their limitations necessitate human oversight, particularly when dealing with sophisticated obfuscation.

5.3 Threats to Validity

We identify several potential threats to the validity of our findings:

Internal Validity

Proxy measurement limitations. Description generation may not fully capture code comprehension. Models could generate plausible

descriptions without understanding algorithms. We mitigate this by also examining deobfuscation performance.

Evaluation subjectivity. Using LLMs to evaluate other LLMs' descriptions introduces potential circularity. We addressed this by using two different evaluation models with clear thresholds and manual verification of samples.

Prompt sensitivity. LLM performance varies with prompt formulation. We used consistent, carefully designed prompts across all models and tasks, though different prompting strategies might yield different results.

External Validity

Dataset characteristics. Our dataset of algorithmic problems, in particular those stemming from coding competitions, may not represent the complexity and diversity of real-world software systems. However, these are extracted from widely-used benchmarks for assessing automated software engineering tools.

Language specificity. Our Java-only focus limits generalizability to languages with different paradigms or structures. However, Java's widespread use, C-family syntax, and object-oriented paradigm make it representative of many mainstream languages. The obfuscation techniques we studied (variable renaming, dead code injection, and literal encryption) are largely language-agnostic and apply similar transformations across multiple languages. While language-specific nuances exist, the fundamental challenges of obfuscation likely transcend specific syntax.

Obfuscation techniques. We used three common techniques rather than an exhaustive set. However, these techniques represent fundamentally different approaches to obfuscation (lexical, structural, and semantic) that target distinct aspects of code understanding. This coverage of the primary obfuscation categories provides insights likely to generalize to variations and combinations of these basic techniques.

Construct Validity

Model selection. Our selection of 13 models, while not exhaustive, strategically covers diverse architectures, parameter scales, and training approaches, including both widely used commercial models and leading open-source alternatives. The consistent patterns observed across this diverse set suggest our findings represent fundamental properties of current LLM technology rather than model-specific behaviors.

5.4 Limitations and Future Directions

Despite the raised threats, the consistent patterns observed in multiple models suggest that our findings are robust. Future research should explore: ① enhancing LLMs' representation learning to reduce dependency on specific naming conventions; ② investigating whether adversarial training with obfuscated code improves robustness; ③ developing hybrid approaches that combine LLM capabilities with traditional program analysis techniques; and ④ establishing more diverse proxies for code understanding beyond description generation. The surprisingly strong performance of general-purpose models suggests that exposure to diverse text patterns during pre-training may be more valuable than narrow code specialization—an insight that could inform future approaches to building more robust code-processing LLMs.

6 CONCLUSION

This study systematically evaluated state-of-the-art LLMs’ ability to process obfuscated code through description generation and deobfuscation tasks. Our findings reveal a critical dependency of LLMs on lexical elements rather than structural understanding. Contrary to expectations, general-purpose LLMs demonstrated greater resilience to obfuscation than code-specialized models. We also identified that all models struggle with concise yet structurally complex code—a blind spot with implications for real-world applications.

We show how code obfuscation serves as an effective challenge task that forces models beyond surface pattern recognition, exposing their reliance on lexical patterns rather than deeper semantic understanding. Developing models that can overcome the obfuscation frontier would represent a substantial advancement in representation learning for code, enabling semantic equivalence detection across syntactic variations and potentially revolutionizing automated program analysis

REFERENCES

- [1] Bertrand Anckaert, Bjorn De Sutter, and Koen De Bosschere. 2007. An entropy estimate of small code modifications. In *2007 IEEE International Conference on Software Maintenance*. IEEE, 15–24.
- [2] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. 2021. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732* (2021).
- [3] Venkatesh Balachandran. 2013. An empirical study of source code metrics for predicting vulnerability in C programs. In *2013 13th International Conference on Quality Software*. IEEE, 103–112.
- [4] Emily M Bender, Timnit Gebru, Angelina McMillan-Major, and Shmargaret Shmitchell. 2021. On the dangers of stochastic parrots: Can language models be too big? *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency* (2021), 610–623.
- [5] Rishi Bommasani, Drew A Hudson, Ehsan Adeli, Russ Altman, Simran Arora, Sydney Von Arx, Michael S Bernstein, Jeannette Bohg, Antoine Bosselut, Emma Brunskill, et al. 2021. On the opportunities and risks of foundation models. *arXiv preprint arXiv:2108.07258* (2021).
- [6] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Pondé de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. In *Advances in Neural Information Processing Systems*.
- [7] Christian Collberg, Clark Thomborson, and Douglas Low. 1997. A taxonomy of obfuscating transformations. *Technical report, Department of Computer Science, The University of Auckland, New Zealand* (1997).
- [8] Damai Dai, Chengqi Deng, Chenggang Zhao, R. X. Xu, Huazuo Gao, Deli Chen, Jia Shi Li, Wangding Zeng, Xingkai Yu, Y. Wu, Zhenda Xie, Y. K. Li, Panpan Huang, Fuli Luo, Chong Ruan, Zhifang Sui, and Wenfeng Liang. 2024. DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models. *arXiv:2401.06066* [cs.CL] <https://arxiv.org/abs/2401.06066>
- [9] DeepSeek-AI. 2025. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. *arXiv:2501.12948* [cs.CL] <https://arxiv.org/abs/2501.12948>
- [10] Sarah Fakhoury, Devjeet Roy, and Venera Arnaudova. 2019. IJA: The Java Identifier Dataset for Research on Identifier Style and Naming. In *Proceedings of the 16th International Conference on Mining Software Repositories*. IEEE, 588–592.
- [11] Chongzhou Fang, Ning Miao, Shaurya Srivastav, Jialin Liu, Ruoyu Zhang, Ruijie Fang, Asmita, Ryan Tsang, Najmeh Nazari, Han Wang, et al. 2023. Large Language Models for Code Analysis: Do LLMs Really Do Their Job? *arXiv preprint arXiv:2305.09129* (2023).
- [12] Dror G Feitelson and Ori Mizrahi. 2020. Comprehension and comprehensibility in the context of program understanding. *Journal of Systems and Software* 163 (2020), 110516.
- [13] Guang Guo, Rongxiang Cai, Shanda Zou, Wei Lu, Shiqi Zhao, Rui Zhang, Atina Wang, Qingying Liu, Zejun Sun, et al. 2021. XCoST: A Benchmark Dataset for Cross-lingual Code Intelligence. In *Findings of the Association for Computational Linguistics: EMNLP 2021*. 4285–4300.
- [14] Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, et al. 2021. Measuring coding challenge competence with apps. *arXiv preprint arXiv:2105.09938* (2021).
- [15] Hamel Husain, Ho-Hsiang Wu, Tiferet Gazit, Miltiadis Allamanis, and Marc Brockschmidt. 2019. CodeSearchNet Challenge: Evaluating the State of Semantic Code Search. In *Proceedings of the Neural Information Processing Systems*.
- [16] Paras Jain, Ajay Jain, Tianjun Zhang, Pieter Abbeel, Joseph E Gonzalez, and Ion Stoica. 2021. Contrastive code representation learning. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. 5954–5971.
- [17] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Yejin Bang, Wenliang Kim, Andrea Madotto, et al. 2023. A Survey of Hallucination in Large Language Models. *arXiv preprint arXiv:2305.10283* (2023).
- [18] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, L  lio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timoth  e Lacroix, and William El Sayed. 2023. Mistral 7B. *arXiv:2310.06825* [cs.CL] <https://arxiv.org/abs/2310.06825>
- [19] Alexandrina Kovacheva. 2013. Analysis of encrypted JavaScript code. In *Information Technology in Innovation Business Conference (ISB-IT’2013)*. 12–14.
- [20] Magnus Lundborg Lagerkvist. 2005. Deobfuscation: Software reverse engineering made simple. *LiU-tryck, Link  ping, Sweden* (2005).
- [21] Alexander LeClair and Collin McMillan. 2019. FunCom: A Large Dataset for Automated Method Documentation Generation. In *Proceedings of the 16th International Conference on Mining Software Repositories*. IEEE, 581–585.
- [22] Anton Lozhkov, Raymond Li, Loubna Ben Allal, Federico Cassano, Joel Lamy-Poirier, Nouamane Tazi, Ao Tang, Dmytro Pykhtar, Jiawei Liu, Yuxiang Wei, Tianyang Liu, Max Tian, Denis Kocetkov, Arthur Zucker, Younes Belkada, Zijian Wang, Qian Liu, Dmitry Abulkhanov, Indraneil Paul, Zhuang Li, Wen-Ding Li, Megan Risdal, Jia Li, Jian Zhu, Terry Yue Zhuo, Evgenii Zheltonozhskii, Nii Osae Osae Dade, Wenhao Yu, Lucas Krau  , Naman Jain, Yixuan Su, Xuanli He, Manan Dey, Edoardo Abati, Yekun Chai, Niklas Muennighoff, Xiangru Tang, Muh-tasham Oblokulov, Christopher Akiki, Marc Marone, Chenghao Mou, Mayank Mishra, Alex Gu, Binyuan Hui, Tri Dao, Armel Zebaze, Olivier Dehaene, Nicolas Patry, Canwen Xu, Julian McAuley, Han Hu, Torsten Scholak, Sebastien Paquet, Jennifer Robinson, Carolyn Jane Anderson, Nicolas Chapados, Mostofa Patwary, Nima Tajbakhsh, Yacine Jernite, Carlos Mu  oz Ferrandis, Lingming Zhang, Sean Hughes, Thomas Wolf, Arjun Guha, Leandro von Werra, and Harm de Vries. 2024. StarCoder 2 and The Stack v2: The Next Generation. *arXiv:2402.19173* [cs.SE] <https://arxiv.org/abs/2402.19173>
- [23] Shuai Lu, Daya Guo, Shuo Ren, Junjie Huang, Alexey Svyatkovskiy, Ambrosio Blanco, Colin Clement, Dawn Drain, Daxin Jiang, Duyu Tang, et al. 2021. CodeXGLUE: A Machine Learning Benchmark Dataset for Code Understanding and Generation. In *Proceedings of the Neural Information Processing Systems*.
- [24] Walid Maalej and Rebecca Tiarks. 2014. How do developers talk about code and why does it matter?. In *2014 IEEE 22nd International Conference on Program Comprehension*. IEEE, 260–270.
- [25] Henry B Mann and Donald R Whitney. 1947. On a test of whether one of two random variables is stochastically larger than the other. *The Annals of Mathematical Statistics* 18, 1 (1947), 50–60.
- [26] Gary Marcus. 2020. The next decade in AI: four steps towards robust artificial intelligence. *arXiv preprint arXiv:2002.06177* (2020).
- [27] Thomas J McCabe. 1976. A complexity measure. *IEEE Transactions on software Engineering* 4 (1976), 308–320.
- [28] Xiaozhu Meng and Barton P Miller. 2013. Effective and efficient control flow measurement with logic analyzability. In *Proceedings of the 2013 International Symposium on Software Testing and Analysis*. 106–117.
- [29] OpenAI. 2023. GPT-4 Technical Report. *arXiv preprint arXiv:2303.08774* (2023).
- [30] OpenAI. 2024. GPT-4o System Card. *arXiv:2410.21276* [cs.CL] <https://arxiv.org/abs/2410.21276>
- [31] David Patterson, Joseph Gonzalez, Urs H  lzle, Quoc Le, Chen Liang, Llu  s-Miquel Munguia, Daniel Rothchild, David So, Maud Texier, and Jeff Dean. 2022. The Carbon Footprint of Machine Learning Training Will Plateau, Then Shrink. *arXiv:2204.05149* [cs.LG] <https://arxiv.org/abs/2204.05149>
- [32] Hammond Pearce, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, and Ramesh Karri. 2022. Asleep at the keyboard? Assessing the security of GitHub Copilot’s code contributions. In *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, 754–768.
- [33] Jiaxin Peng and Hongyang R Zheng. 2021. Could machine learning be a general pattern? transcending the disciplines would be revolutionary. *AI & SOCIETY* 36 (2021), 977–982.
- [34] Mila Dalla Preda and Roberto Giacobazzi. 2015. Identifying metamorphic virus variants using control and data flow analysis. In *2015 IEEE 14th International Conference on Machine Learning and Applications (ICMLA)*. IEEE, 126–132.
- [35] Nicolas Prenner and Romain Robbes. 2021. Automatic detection of code smells: A comparison and evaluation of classical ML and deep learning. In *2021 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 328–339.
- [36] Ruchir Puri, David Kung, Geert Janssen, Wei Zhang, Giacomo Domeniconi, Vladimir Zolotov, Julian Dolby, Jie Chen, Moninder Choudhury, Lindsey Decker, et al. 2021. Project CodeNet: A Large-Scale AI for Code Dataset for Learning a

- Diversity of Coding Tasks. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*.
- [37] Ruchir Puri, David S. Kung, Geert Janssen, Wei Zhang, Giacomo Domeniconi, Vladimir Zolotov, Julian Dolby, Jie Chen, Mihir Choudhury, Lindsey Decker, Veronika Thost, Luca Buratti, Saurabh Pujar, Shyam Ramji, Ulrich Finkler, Susan Malaika, and Frederick Reiss. 2021. CodeNet: A Large-Scale AI for Code Dataset for Learning a Diversity of Coding Tasks. *arXiv:2105.12655 [cs.SE]* <https://arxiv.org/abs/2105.12655>
- [38] Veselin Raychev, Martin Vechev, and Andreas Krause. 2015. Predicting program properties from big code. In *Proceedings of the 42nd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. 111–124.
- [39] Marco Tulio Ribeiro, Tongshuang Wu, Carlos Guestrin, and Sameer Singh. 2020. Beyond accuracy: Behavioral testing of NLP models with CheckList. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. 4902–4912.
- [40] Baptiste Rozière, Yossi Adi, Ivan Vulić, Jan Botha, Utku Evci, Anjana Goyal, Alexis Passos, Cyprien D'Autume, Somayeh Haghighi, Vegard Pedro, et al. 2023. Code Llama: Open Foundation Models for Code. *arXiv preprint arXiv:2308.12950* (2023).
- [41] Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, Jérémy Rapin, Artyom Kozhevnikov, Ivan Evtimov, Joanna Bitton, Manish Bhatt, Cristian Canton Ferrer, Aaron Grattafiori, Wenhan Xiong, Alexandre Défossez, Jade Copet, Faisal Azhar, Hugo Touvron, Louis Martin, Nicolas Usunier, Thomas Scialom, and Gabriel Synnaeve. 2024. Code Llama: Open Foundation Models for Code. *arXiv:2308.12950 [cs.CL]* <https://arxiv.org/abs/2308.12950>
- [42] Margaret-Anne Storey. 2006. The tools programmers use. *Computer* 39, 3 (2006), 46–52.
- [43] superblabeere27. 2025. Obfuscator. <https://github.com/superblabeere27/obfuscator> Accessed: 12 March 2025.
- [44] Alexey Svyatkovskiy, Shao Kun Deng, Shengyu Fu, and Neel Sundaresan. 2020. IntelliCode compose: Code generation using transformer. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 1433–1443.
- [45] CodeGemma Team, Heri Zhao, Jeffrey Hui, Joshua Howland, Nam Nguyen, Siqi Zuo, Andrea Hu, Christopher A. Choquette-Choo, Jingyue Shen, Joe Kelley, Kshitij Bansal, Luke Vilnis, Mateo Wirth, Paul Michel, Peter Choy, Pratik Joshi, Ravin Kumar, Sarmad Hashmi, Shubham Agrawal, Zhitao Gong, Jane Fine, Tris Warkentin, Ale Jakse Hartman, Bin Ni, Kathy Korevec, Kelly Schaefer, and Scott Huffman. 2024. CodeGemma: Open Code Models Based on Gemma. *arXiv:2406.11409 [cs.CL]* <https://arxiv.org/abs/2406.11409>
- [46] Gemma Team. 2024. Gemma. (2024). <https://doi.org/10.34740/KAGGLE/M/3301>
- [47] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. LLaMA: Open and Efficient Foundation Language Models. *arXiv preprint arXiv:2302.13971* (2023).
- [48] Michele Tufano, Dawn Drain, Alexey Svyatkovskiy, and Neel Sundaresan. 2022. Using large language models to enhance programming error messages. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 380–391.
- [49] Michele Tufano, Colin Watson, Gabriele Bavota, Massimiliano Di Penta, Martin White, and Denys Poshyvanyk. 2022. Learning to generate code explanations from code snippets, identifiers, and dependencies. *IEEE Transactions on Software Engineering* 49, 1 (2022), 215–231.
- [50] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).
- [51] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. 2022. Chain of thought prompting elicits reasoning in large language models. *arXiv preprint arXiv:2201.11903* (2022).
- [52] Yuxiang Wei, Federico Cassano, Jiawei Liu, Yifeng Ding, Naman Jain, Zachary Mueller, Harm de Vries, Leandro von Werra, Arjun Guha, and Lingming Zhang. 2024. SelfCodeAlign: Self-Alignment for Code Generation. *arXiv:2410.24198 [cs.CL]* <https://arxiv.org/abs/2410.24198>
- [53] Frank F Xu, Uri Alon, Graham Neubig, and Vincent J Hellendoorn. 2022. A systematic evaluation of large language models of code. *arXiv preprint arXiv:2202.13169* (2022).
- [54] An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, Guanting Dong, Haoran Wei, Huan Lin, Jialong Tang, Jialin Wang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Ma, Jianxin Yang, Jin Xu, Jingren Zhou, Jinze Bai, Jinzheng He, Junyang Lin, Kai Dang, Keming Lu, Keqin Chen, Kexin Yang, Mei Li, Mingfeng Xue, Na Ni, Pei Zhang, Peng Wang, Ru Peng, Rui Men, Ruize Gao, Runji Lin, Shijie Wang, Shuai Bai, Sinan Tan, Tianhang Zhu, Tianhao Li, Tianyu Liu, Wenbin Ge, Xiaodong Deng, Xiaohuan Zhou, Xingzhang Ren, Xinyu Zhang, Xipin Wei, Xuancheng Ren, Xuejing Liu, Yang Fan, Yang Yao, Yichang Zhang, Yu Wan, Yunfei Chu, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, Zhifang Guo, and Zhihao Fan. 2024. Qwen2 Technical Report. *arXiv:2407.10671 [cs.CL]* <https://arxiv.org/abs/2407.10671>
- [55] Yonghao Yang, Tianshu Zhu, Xuehai Fu, Jackie Wang, Zijian Zhang, Jiaying Liu, Yuqian Wang, and Caiwen Xiong. 2022. A taxonomy of prompt modifiers for text-to-image generation. *arXiv preprint arXiv:2204.13988* (2022).
- [56] Junjie Ye, Xuanning Chen, Nuo Xu, Can Zu, Zekai Shao, Shichun Liu, Yuhua Cui, Zeyang Zhou, Chao Gong, Yang Shen, Jie Zhou, Siming Chen, Tao Gui, Qi Zhang, and Xuanjing Huang. 2023. A Comprehensive Capability Analysis of GPT-3 and GPT-3.5 Series Models. *arXiv:2303.10420 [cs.CL]* <https://arxiv.org/abs/2303.10420>
- [57] Zhengran Zhao, Zejian Shi, Yichen Fu, Junjie Yu, Zhenchang Li, and Tao Xu. 2021. Recent Advances in Programming Language Processing: Research Contributions from the LeetCode Domain. *arXiv preprint arXiv:2108.13324* (2021).